

Bad Poets Society - Process and Reflections

Ranam Hamoud and Yuning Yao and Godwin Addetsi

Introduction

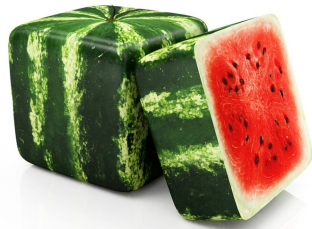


Figure 1: Square watermelon, thought it was fitting...

A symmetric square poem is a grid of words that can be read both across and down, with perfect symmetry. Whatever word appears at position (i, j) also appears at position (j, i) . The form is rare because the constraints are severe: every line has to read as coherent poetry, and the underlying structure has to fit this exact pattern.

We set out to build a system that could write these poems automatically. The task turned out to be much harder than we expected. Recent language models write strong, expressive poetry, but they do not reliably keep track of global constraints such as symmetry across an entire grid.

Our solution was to treat the problem as a collaboration between creative generation and explicit constraint solving. Instead of asking the model to solve the puzzle on its own, we asked it for candidate words and short lines, then used a classical backtracking algorithm to search through combinations and check the rules in code. In what follows, we describe early attempts that failed, the moment we reframed the task as a search problem, and the version that finally worked for grids up to 6×6 on themes like the Seven Deadly Sins that we adopted. We then evaluate the outputs with Ritchie’s criteria for computational creativity and reflect on what we learned about square poems, language models, and constraint-based creativity. By the end of the project, we felt a bit like square watermelons ourselves, grown in a box of constraints, and still expected to be juicy.

Inspiring Set

We collected an inspiring set of historical and mathematical examples of word squares and square poems, with a special focus on Lewis Carroll.

The SATOR word square

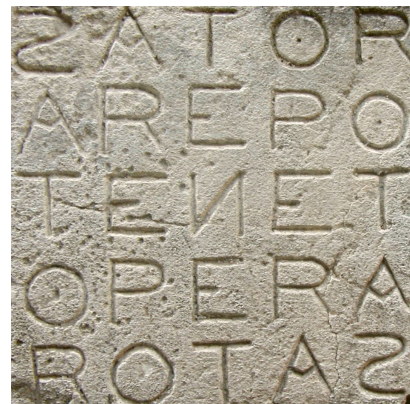


Figure 2: A classic Sator square etched onto a wall.

The Sator square is a 5×5 Latin word square with multiple symmetries. It is square in both geometry and content, has diagonal and 180° rotational symmetry, and can be read top to bottom, bottom to top, left to right, and right to left (Christie, 2021). It is often cited as an early example of text that is both linguistic and combinatorial at the same time (Birken and Coon, 2008). This example pushed us to think of square poems not just as lines of text but as objects with strong structural rules.

Lewis Carroll’s grid poem and style

The core inspiration for our system is the 6×6 grid poem attributed to Lewis Carroll. This poem forms an array with six lines of six words, constructed so that the same sequence of words can be read across and down (Christie, 2021). It is a symmetric poem that can be read horizontally and vertically, and it comments on the difficulty of constructing such a structure. In other words, it is not a simple poem but a carefully designed English word square in which both horizontal and vertical readings are grammatically coherent.

I	often	wondered	when	I	cursed,
Often	feared	where	I	would	be—
Wondered	where	she'd	yield	her	love,
When	I	yield,	so	will	she.
I	would	her	will	be	pitied!
Cursed	be	love!	She	pitied	me ...

Figure 3: Lewis Carroll’s 6×6 square poem layout.

Lewis Carroll (Charles Lutwidge Dodgson) was not only a writer of unusual literature but also a mathematician and logician at Christ Church, Oxford. His work often mixes mathematical structure with playful language (Abeles, 2015). That balance of strict form and whimsical content became our target.

The more we looked at Lewis Carroll’s work, the more fascinating it became. His most famous book, *Alice’s Adventures in Wonderland*, had always felt like a weird, dreamlike story to us. During this project, we learned that it is also a clever satire of shifts in algebra and geometry at the time. A New York Times article suggests that when the Caterpillar warns Alice about changing size, this reflects confusion around symbolic algebra (using x and y) (Bayley, 2010). In traditional arithmetic, numbers have fixed values. In the newer algebra, values could change, like Alice.

We also revisited his poem “Jabberwocky.” It looks like nonsense, but it follows strict grammatical rules. Words like “vorpal” and “slithy” do not appear in any dictionary, but our brains still try to assign them meaning based on rhythm and syntax (Alice-in-Wonderland.net). This is close to how language models behave, and it reinforced our interest in forms where structure does a lot of hidden work.

After this deep dive, we felt that Carroll’s grid poem in Figure 3 captured the balance we wanted: romantic, slightly melodramatic content embedded in a rigid mathematical frame. The poem behaves as a puzzle-like object with clear rules, but it also reads as a poem. In designing our system, we took this poem as both a stylistic and structural prototype.

Further historical square poems and shaped verse

Larger historical square poems exist, such as Henry Lok’s 10×10 Elizabethan poem in honor of Queen Elizabeth I, where each of the ten lines consists of ten single-syllable words and different routes through the square reveal embedded sub-poems (Christie, 2021). These examples showed us how far the idea can be pushed.

Geometric forms, including the square, can act as organizing principles for verse (Birken and Coon, 2008). Shaped and grid-based poems treat the page as a space to structure meaning. This encouraged us to treat our generated poems as geometric objects, where visual form and combinatorial constraint matter as much as lexical content.

This documented inspiring set guided our prompt design and later gave us something concrete to compare against when

we looked at our own outputs.

Development Process and Failures

In this section we describe the system more as a “build diary” than a final design. Each phase reflects where we were at that point.

The Challenge

Our goal was to build a system that generates square poems: poems arranged in an $N \times N$ grid where each row reads as a coherent line of poetry, and the grid is symmetric. This means the word at position (i, j) is the same as the word at position (j, i) . This creates an interesting constraint: the diagonal words form a kind of spine for the poem, and every word placement affects both its row and its corresponding column.

At first, this did not seem too scary. Modern language models are very good at writing poetry. It was tempting to assume that the right prompt would be enough.

Phase 1: Asking the LLM Directly

Our first attempt was as direct as possible. We gave the language model a detailed prompt that explained what a square poem is, provided a theme such as “wrath” or “sloth”, specified the grid size, and asked it to generate the complete poem. The prompt described the symmetry rule, gave an example of the desired layout, and even reminded the model that position (i, j) must match position (j, i) for all cells.

The results were pretty to read but structurally wrong. The model produced thematically consistent, sometimes striking poetry, yet the symmetry constraint almost never held. In a 5×5 grid, there are 10 pairs of cells that must match, plus 5 diagonal cells. The model might get 6 or 7 pairs correct, but there were always a few mismatches.

It was even more confusing when we asked the model to check its own output. It often claimed that the grid was fully symmetric. When we pointed to specific violations, it apologized and regenerated, and the new poem usually had different violations.

For us, this was the first clear sign that the model did not keep a global scratch space for this kind of structure. It generates text token by token. It is very good at local coherence, so each next word fits the previous ones. It is much weaker at constraints that require comparing distant positions in the output. No amount of prompt tweaking changed that. At this point we realized that the architecture itself was not designed for exact constraint satisfaction.

Phase 2: Row-by-Row Generation

Since the model struggled with the whole grid at once, we turned to a more incremental strategy. We asked the model to generate the first row (5 words for a 5×5 grid), then we copied that row into the first column in code to enforce symmetry. For the second row, we fixed the first word in advance (to match the earlier column) and asked the model to complete the rest of the row. We repeated this pattern down the grid.

This step already did better. Symmetry moved out of the model and into our code, so the grid could not violate that rule.

New issues quickly showed up. The model often reused words from earlier rows, so poems were repetitive. Sometimes the forced prefix at the start of a row was awkward, and the model struggled to complete it into grammatical English. If a later row became impossible to complete, the system had no way to recover. It simply failed and returned nothing.

We learned two lessons here. First, symmetry is easier to handle in code than in prompting. Second, the system needed some notion of validation and backtracking. It had to be able to reject bad rows and revisit earlier choices.

Phase 3: Reframing as a Search Problem

At this point, we started to talk about the task in a different way. Generating a square poem felt less like pure text generation and more like a search problem. We were looking for a configuration of words that satisfies several constraints at once:

- symmetry across the main diagonal,
- uniqueness of words and word roots,
- grammatical coherence of each row,
- thematic cohesion for the whole grid,
- a sense of closure in the final row.

This sounded exactly like a constraint satisfaction problem. Classical AI already has tools for that, in particular backtracking search.

We decided that the language model would act more like a creative friend that suggests candidate lines, and the search algorithm would act like a strict editor that checks the rules. Before implementing the search, we needed practical constraint checks.

Word uniqueness turned out to be trickier than it looked. The words “run”, “runs”, “running”, and “ran” are different strings, but using all of them in a short poem feels repetitive. We wanted uniqueness at the level of lemmas, not surface forms. For this, we brought in Stanza, a pretrained NLP pipeline (Qi et al., 2020), and used its lemmatizer.

For grammar, we experimented with part-of-speech tagging. For example, we decided that rows that end in articles or prepositions feel incomplete and should be rejected. For naturalness, we used perplexity with GPT-2, a pretrained language model (Radford et al., 2019). Perplexity here is a rough measure of how surprising the text is to the model. We treated very high perplexity lines as awkward and filtered them out.

Finally, we added a specific check for the last row: it has to contain at least one noun and one verb. This is inspired by our inspiring set, where Carroll’s poem ends with a complete sentence rather than a fragment.

Phase 4: The Backtracking System

The final system uses three main components: constraint validation, domain generation, and search.

Constraint Validation. Each candidate row goes through several checks. First, we lemmatize the words with Stanza

(Qi et al., 2020). This lemmatization means that “run” and “running” are recognized as the same word, so the system enforces semantic uniqueness rather than only comparing surface strings.

Next, we check grammatical constraints using part-of-speech tags. Rows that end with articles or prepositions are rejected because they feel unfinished. We also compute perplexity with GPT-2 (Radford et al., 2019). Candidates with very high perplexity are treated as unnatural and are discarded. We chose GPT-2 because it was trained on diverse internet text and has broad coverage of English constructions, and it is still computationally light enough for repeated checks inside a search loop.

For the final row, an extra constraint requires the presence of both a noun and a verb, so the poem concludes with a complete thought. This reflects our inspiring set: Carroll’s square poem ends with a full sentence that gives the grid a sense of closure.

Domain Generation. Candidate rows come from LLaMA, running locally through Ollama, which we picked for its creative writing behavior and to avoid API costs during development (AI@Meta, 2024). The prompt changes based on context. For the first row, we ask for a thematically rich line with exactly N words. For later rows, the prompt includes all rows so far, the words already locked in by symmetry, and a list of forbidden words.

If candidates keep failing validation for a given row, we gradually increase the model’s temperature. This tends to make the outputs less obvious and can help us escape small, repetitive regions of the search space.

Search and Stopping Criteria. The search starts with the first row. The system generates several candidates, filters them through the validator, and commits the first candidate that passes all checks. As soon as a row is accepted, the corresponding column is filled by symmetry.

The search then moves to the next row. Each new row already contains some fixed words from previous symmetry constraints. By the last row of a 5×5 grid, four of the cells in that row are fixed and only one remains free.

We use four stopping criteria:

- **Success:** All N rows are filled and all constraints are satisfied, so the system returns the completed poem.
- **Backtrack limit:** If the number of backtracks exceeds 200, the system stops. We picked this limit experimentally. Lower values failed too often on difficult themes, and much higher values wasted time.
- **Generation exhaustion:** If the language model cannot produce any valid candidate for a row after several attempts (with temperature adjustments), the system reports failure.
- **Invalid configuration:** If the user provides invalid parameters, such as a bad grid size, the system exits early with an error message.

Our system is a fixed-length generator. The user chooses N , and the output is always an $N \times N$ grid. This fits the nature of square poems. The dimension is part of the identity of

the form, in the same way that Carroll’s 6×6 layout is not accidental.

Empirical Observations

During generation runs, the system usually samples 8 candidates per row and then filters them. The most common rejection reasons we saw in the logs were:

- **Lemma collisions:** a word with the same root as an earlier word appears again, for example “whisper” and “whispered”,
- **Direct repetition:** an exact word from an earlier position appears a second time,
- **Bad endings:** the row ends in an article (“the”, “a”) or a preposition (“to”, “in”).

The system successfully generates 4×4 , 5×5 , and 6×6 grids. Larger grids are possible in principle, but they become much harder in practice because later rows have long fixed prefixes. By the final row of a grid, all but one of the words in the row are already fixed by symmetry, so the system only has to generate a single remaining word that obeys all constraints.

Example Outputs

To check that the system really works and is not just solving a single hand-picked case, we generated poems for all seven sins across multiple runs and grid sizes (from 4×4 up to 6×6). In each case, the final outputs respect the symmetry constraint and pass our grammatical and uniqueness filters. At a high level, Wrath tends to converge on imagery of fire, storms, and violent motion. Sloth drifts toward soft textures, slow time, and sleep. Greed and Gluttony, especially in 6×6 form, often feel crowded and heavy, with repeated references to ghosts, halls, banquets, and hunger. These patterns reflect both the language model’s training data and the pressure of the square constraint.

A full selection of generated grids for all seven themes is included in the Appendix, where each poem is formatted as an explicit $N \times N$ table.

AI-Powered Poem Presentation

Presentation Concept and Design

Once we had poems that worked on paper, we wanted to see how far we could push the presentation. For our final demo, we turned the generated square poems into an audiovisual sequence that makes both their structure and mood easier to see.

Each poem was tied to one of the Seven Deadly Sins, so we designed visual and audio layers that matched each sin and kept the square layout visible at all times.

Visual Presentation Implementation

We used Google Gemini (Veo 3.1) and Kling AI VIDEO 2.5 Turbo to generate short cinematic clips from detailed prompts that described each sin. For example, Lust used burning red velvet and soft light, Envy used green, reptilian eyes and glass, and Gluttony used rotting baroque banquets.

The poems appeared as square grids over these clips. During narration, we highlighted the active row and the matching column with red outlines. This made the symmetry obvious even for viewers who did not know the rule in advance.

Audio Presentation Implementation

For narration, we used ElevenLabs to pick voices that matched each sin. Lust had a warm, low voice. Envy had a tense, whispery voice. Gluttony had a slower, heavier voice. We repeated this process for Wrath, Sloth, Greed, and Pride. This voice casting step sounds small, yet it made a difference. Each poem gained its own identity, and the connection between theme and sound became clearer.

Creative Curation and Refinement

The final presentation combined AI-generated video, AI voices, and our animated grid overlay. In practice, this did not feel automatic at all. We spent time discarding clips that did not match the mood, adjusting timings, and tweaking line breaks so that the visual and spoken rhythm matched.

For us, this stage confirmed something important. Generative AI was most useful as a flexible medium. It gave us many options for images and voices, yet we still had to direct, select, and arrange. The presentation also helped non-technical viewers understand what is special about the poems, because it put the structure and emotion in one place.

Reflection & Evaluation

Evaluating Creativity through Ritchie’s Criteria

To connect our project to computational creativity theory, we used Ritchie’s criteria (Ritchie, 2006). All three team members reviewed the poems and discussed them under four headings: typicality, novelty, surprise, and quality.

Typicality. The outputs look and feel like poems. They use thematic language such as “pillowy silence” for Sloth, “stormy frenzy” for Wrath, and “emerald fleeting whispers” for Envy. Each row reads as a phrase that a human might write. The square layout is unusual, yet core poetic elements such as imagery, condensed language, and shared theme are present.

Novelty. Symmetric square poems do not appear often in human poetry. The structure itself is already unusual. On top of that, the system produced multiple valid poems for the same theme with different wording. For Wrath, we saw both “fury clouds mind with scarlet” and “rage paints the world crimson” as valid solutions in different runs. This suggests that the system is exploring a space of possibilities rather than always landing on the same answer.

Surprise. A lot of the surprise comes from interactions between rows and columns. A word that completes a row also appears down the column, so it picks up extra neighbors and meaning. For example, in the Wrath poem, a word like “fury” can appear in a position that links it to both “scarlet” and “frenzy”. These connections were not planned by us and not fully planned by the model either. They emerge from the symmetry rule combined with the search.

Quality. Our constraints filter out extreme cases: ungrammatical lines, repeated words, and very high perplexity lines

that sound off. Even with this, the poems vary in smoothness. The Sloth poem (“time melts into soft whispers / melts like butter under sun”) reads in a natural way. Some Envy lines (such as “a deep fleeting green sighs”) feel more experimental or slightly awkward. The 6×6 poems, such as Greed with lines like “hungry ghosts haunt in shadows unseen”, show that the system can scale up and keep coherence for larger grids, although it takes longer.

Honest Assessment of Creative Limitations

Our system clearly introduces structure and novelty, yet it is important to be honest about what kind of creativity it shows. The content of each line comes entirely from the language model. Our code does not invent new metaphors. It selects and arranges already creative fragments under strict rules. In that sense, a lot of the semantic creativity belongs to the underlying LLM.

Our main creative contribution sits in the formal dimension. We use a square constraint that is rare in human poetry and combine it with a hybrid pipeline that most poets do not have. The system is creative in how it searches a constrained space and in how it turns a general-purpose model into a specialist on square poems.

We also noticed that some themes, such as Wrath, naturally give richer imagery. The model has many words for anger, fire, and destruction. Other themes, such as Envy, seem to pull from a narrower set of phrases. This creates an uneven feel between poems and suggests that the training data shapes our results more than we might like.

Under Ritchie’s framework, we think the system does show computational creativity, especially on typicality, novelty, and surprise. The quality dimension is more mixed and sensitive to the theme and grid size.

Comparison to Carroll’s Original

A direct comparison to our inspiring set highlights both what we achieved and what we missed.

Carroll’s poem (“I often wondered when I cursed / Often feared where I would be...”) manages something our system does not: it reflects on its own form. The poem talks about the difficulty of writing such a square at the same time as it instantiates that square. Lines such as “Wondered often my lot cursed” can be read as comments on the strain of composition, and that meta-layer sits inside the rigid grid.

Our outputs are thematically coherent, yet they do not reach this kind of self-awareness. When our system generates a poem about Wrath, it produces images of anger and fire, but it does not talk about the act of constrained writing itself. This gap suggests a direction for future work, where prompts deliberately ask for square poems that comment on their own construction.

Carroll’s word choices also feel more deliberately odd. Phrases like “lot cursed” or “feared where” create tension and ambiguity that invite interpretation. Our system, which filters candidates using perplexity, tends to favor more expected phrases. Occasionally accepting candidates with higher perplexity might move our poems closer to Carroll’s strange, slightly imbalanced tone.

Limitations and Lessons Learned

Looking back at the project, several practical limits and missed opportunities stand out.

On the process side, we only framed the task as a constraint satisfaction problem after spending a fair amount of time on prompt tweaking. In hindsight, we would start from the CSP framing and treat direct prompting as a baseline instead of a main strategy. We also did not keep a structured log of rejected candidates across runs. The system prints rejections, but we did not collect them for analysis. A simple log file or spreadsheet would have helped us see patterns and tune constraints more systematically.

We also tested most of our pipeline on a small set of themes at first. Only later did we try all seven sins. This meant that we discovered vocabulary differences between themes relatively late in the project.

On the technical side, the current implementation runs into practical limits around 13×13 grids. The search space grows fast, and our sequential calls to the language model make long runs slow. The system is also tied to English. We rely on English part-of-speech tags, English lemmatization, and an English language model. The generator does not model rhythm or syllable counts, so we do not control meter like in poems.

These limitations are not surprising for a first version, but they are useful to keep in mind for future work.

Future Directions

- **Constraint Exploration:** Extend the system to other formal constraints such as palindromic rows, acrostics, or embedding simple numerical sequences in the grid. Each new constraint would create a different kind of puzzle-poem.
- **Human-AI Co-Creation:** Build an interactive interface where a human can lock in some words, relax certain constraints, or nudge the theme, then let the system fill in the rest.
- **Meta-Creativity:** Experiment with prompts that explicitly ask for poems about constraints and composition, in order to move closer to Carroll’s self-aware style.
- **Multi-lingual Support:** Port the pipeline to other languages by swapping in local language models and NLP tools.

Conclusion

The square poem generator shows that a hybrid of constraint solving and language modeling can produce a small but interesting corner of computational poetry. The symmetry requirement, which looks restrictive at first, actually creates new links between words and introduces a playful, puzzle-like quality to the poems.

The main lesson for us is that structure and creativity can support each other. The language model brings rich, thematic language. The constraint solver brings discipline and precision. Together they can do something that neither part can manage alone.

We also found that presentation matters. By turning the poems into an audiovisual sequence with grids, video clips,

and voices, we made the underlying structure visible and gave each poem a stronger emotional tone. This helped us and our audience understand what the system was doing. This project forced us to connect ideas from natural language processing, classical AI, and computational creativity in a concrete way. It also reminded us that failure paths, such as direct prompting that does not work, are useful data points. They shape the final design as much as the successes.

References

- Abeles, F. F. 2015. Mathematics: Logic and lewis carroll. *Nature* 527(7578):302–304.
- AI@Meta. 2024. The LLaMA 3 Herd of Models. Technical report, Meta AI.
- Alice-in-Wonderland.net. Jabberwocky – poem origins. <https://www.alice-in-wonderland.net/resources/analysis/poem-origins/jabberwocky/>.
- Bayley, M. 2010. Algebra in wonderland. *The New York Times*. Opinion.
- Birken, M., and Coon, A. C. 2008. *Discovering Patterns in Mathematics and Poetry*. Amsterdam: Brill Academic Publishers.
- Christie, M. 2021. Square poems. In *From Fibs to Fractals: Exploring Mathematical Forms in Poetry*. Beir Bua Press. Chapter PDF available via Marian Christie, “Square Poems” <https://marianchristiepoetry.net/>.
- Qi, P.; Zhang, Y.; Zhang, Y.; Bolton, J.; and Manning, C. D. 2020. Stanza: A python nlp package for many human languages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 101–108.
- Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; and Sutskever, I. 2019. Language models are unsupervised multitask learners. Technical report, OpenAI.
- Ritchie, G. 2006. A taxonomy of creative systems. *Knowledge-Based Systems* 19(7):449–455.

Appendix: Generated Poems

The following poems are some of the generated ones by our system across the seven deadly sins themes.

rage	paints	the	world	crimson
paints	your	vision	scarlet	fury
the	vision	dances	in	fire
world	scarlet	in	stormy	frenzy
crimson	fury	fire	frenzy	erupts

Wrath (5×5)

time	melts	into	soft	whispers
melts	like	butter	under	sun
into	butter	dreams	of	clouds
soft	under	of	pillowy	silence
whispers	sun	clouds	silence	embrace

Sloth (5×5)

pride	stands	unwavering	against	doubt
stands	resolute	in	its	resolve
unwavering	in	steadfast	truth	unbending
against	its	truth	unyielding	grit
doubt	resolve	unbending	grit	remains

Pride (5×5)

envy	paints	eyes	a	shade
paints	aura	of	deep	jealousy
eyes	of	emerald	fleeting	whispers
a	deep	fleeting	green	sighs
shade	jealousy	whispers	sighs	bittersweet

Envy (5×5)

burning	passions	devour	innocence	without	remorse
passions	consume	hearts	in	silent	despair
devour	hearts	yearning	entwines	fates	unseen
innocence	in	entwines	shadows	dance	eternally
without	silent	fates	dance	desire's	chill
remorse	despair	unseen	eternally	chill	whispers

Lust (6×6)

hungry	ghosts	haunt	in	shadows	unseen
ghosts	drift	through	darkened	twilight	halls
haunt	through	gluttonous	whispers	beneath	despair
in	darkened	whispers	crimson	hues	linger
shadows	twilight	beneath	hues	silent	yearning
unseen	halls	despair	linger	yearning	grasping

Greed (6×6)

hunger	devours	every	living	moment
devours	each	delicious	morsel	whole
every	delicious	craving	consumes	myself
living	morsel	consumes	taste	eternally
moment	whole	myself	eternally	surrendered

Gluttony (5×5)